Project 1: Network Deployment, Measurement, and Observability.

This project examines the deployment and operation of a software-based telecommunications network from the perspective of a network engineer. The objective is to establish an operational communication path, determine how traffic and network functions behave across the system, quantify service performance under controlled conditions, and use packet-level, system-level, and application-level evidence to explain normal operation, degradation, and failure. The project is centered on the engineering workflow **Deploy → Verify → Observe → Measure → Diagnose → Explain** and establishes the practical foundation of network deployment, measurement, and observability for the remainder of the course.

Scenario A: Software-Based Telecommunications Network

Reference Architecture: **UE / Client → Access Network → Core Network / Gateway → Data Network → Application Server**

Scenario Overview: Scenario A represents a simplified software-based telecommunications environment in which each functional stage of the communication path is explicitly observable and measurable. The objective is not to reproduce a complete 4G/5G network, but to establish a technically meaningful end-to-end telecommunications environment in which students can investigate how packets move through multiple functional network stages and how network conditions affect service behavior. The baseline implementation uses a single Linux virtual machine and Linux network namespaces to represent the principal network entities. Virtual Ethernet (veth) pairs are used to interconnect the namespaces. Linux network namespaces provide isolated network interfaces, routing tables, sockets, and other network state, while veth pairs provide virtual point-to-point links between namespaces. To introduce containerized network services without hiding the underlying Linux networking concepts, Scenario A adopts a Hybrid Linux + Docker architecture:

Linux namespaces provide the UE, Access Router, and Core/Gateway networking functions.

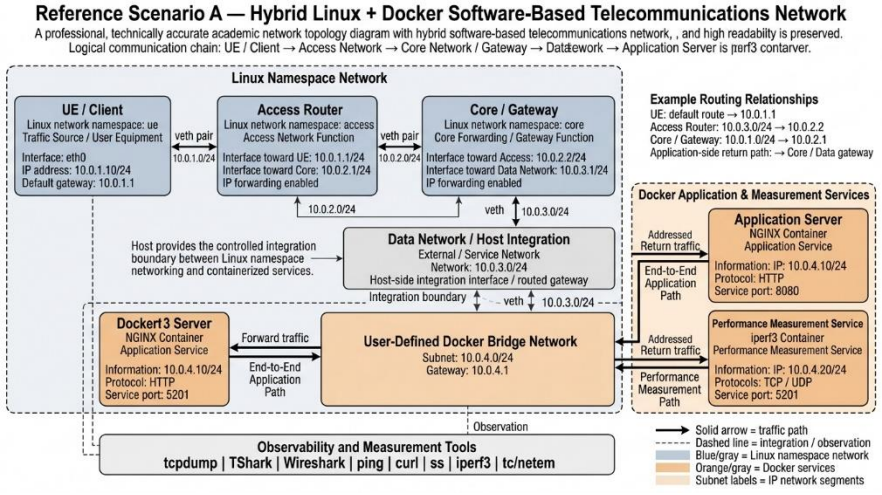
A Docker-based application server provides the application service.

A Docker-based iperf3 service may be used as a network performance measurement endpoint.

A Docker user-defined bridge network provides a controlled containerized extension of the Data Network.

The Linux host provides the integration boundary between the namespace-based network and the containerized services.

The hybrid architecture therefore preserves the original communication path while introducing a realistic software-deployment mechanism: Linux network infrastructure + containerized network/application functions. A recommended baseline topology is shown below (take it as a reference!). The official Nginx Docker image is maintained as a Docker Official Image and provides standard Nginx server images for containerized web services. The networkstatic/iperf3 image is specifically designed for network performance and bandwidth testing and provides the iperf3 client/server functionality. The current image provides both amd64 and arm64 variants.



The hybrid architecture therefore preserves the original communication path while introducing a realistic software-deployment mechanism: Linux network infrastructure + containerized network/application functions. A recommended

Step 1: Network Deployment and End-to-End Operation

Given the reference architecture, establish the network environment and demonstrate successful end-to-end communication between the UE/client and the application server. You shall identify and document the principal components, network interfaces, IP addresses, subnet relationships, routing entries, transport-layer ports, services, and communication relationships involved in the complete path. Your analysis shall explain how application traffic traverses the reference architecture and shall identify the observations that demonstrate that the complete communication path is operational. The objective is not merely to demonstrate that a ping command succeeds. You must establish that the relevant network interfaces, routes, services, and application endpoints are correctly configured and that traffic can traverse the complete communication path. Recommended implementation approach:

Start with a single Linux VM and construct the basic network using four network namespaces: ue, access, core, app, etc. Create the namespaces using ip netns, then construct the topology with three virtual Ethernet pairs. Linux network namespaces isolate network state, while veth devices provide interconnected virtual Ethernet endpoints suitable for connecting separate namespaces. You may show topology diagram; namespace list; interface configuration; IP addressing table; routing table output; Docker network configuration; container/service information; service status; application listening-port information; successful curl or equivalent application test; relevant screenshots or command output; short explanation of the end-to-end communication path.

Step 2: Traffic Observation and Protocol Behavior

Using the established communication path, analyze representative ICMP, TCP, UDP, and application traffic at the packet level. For each traffic type, identify: source address; destination address; protocol; transport protocol, where applicable; source and destination port; protocol hierarchy; packet sequence; selected timing information; relevant flags, sequence numbers, acknowledgements, or other protocol fields; evidence of successful or unsuccessful communication. Your analysis shall explain how the communication process appears in the packet trace and how packet-level evidence can be used to determine whether communication is functioning correctly. Required traffic categories: A. ICMP traffic (focus on packet size; observed RTT; packet loss, if any); B. TCP traffic (Generate TCP traffic through either: HTTP → Nginx (identify: TCP connection; HTTP request; HTTP response) or iperf3 TCP.); C. UDP traffic (use iperf3 or another controlled UDP source); D. Application traffic. Packet capture: use one or more of: tcpdump, tshark, or Wireshark. Comparing captures at different locations can help determine whether a packet was generated; a packet was forwarded; a packet was dropped; a response was generated; a response was successfully returned.

Step 3: Performance Measurement and Network Degradation

Establish a quantitative performance baseline for the reference network and repeat the experiment under at least two controlled degraded network conditions. Examples include increased delay; packet loss; bandwidth limitation; combinations of these conditions. Compare the resulting performance using appropriate quantitative metrics. Possible metrics include: RTT; TCP throughput; UDP throughput; packet loss; retransmission count; jitter; application response time; connection establishment time; service availability. The analysis shall explain not only what changed, but why the change occurred, using both quantitative results and supporting packet-level or system-level evidence. Before introducing any impairment, document the normal operating condition. For example: Condition: Baseline / no artificial impairment, then measure: ICMP RTT; TCP throughput; UDP throughput; UDP loss/jitter; HTTP response time and TCP retransmissions. Repeat measurements sufficiently to determine whether the result is reasonably stable. You can use Linux tc and the NetEm mechanism to introduce controlled network conditions. Potential experiments include increased delay, 1% Packet loss, 20 Mbps limit, etc. You may show the experimental topology; a list of experimental conditions; commands/configuration used to impose impairment; raw measurement output; original packet captures where appropriate; at least one comparative table; at least one suitable plot; quantitative comparison; interpretation of the results; or an explanation linking network conditions to observed protocol behavior.

Reference Scenario B: Advanced Software-Based 5G Telecommunications Network

Reference Architecture:

UE → 5G RAN / gNB → 5G Core / UPF → N6 Data Network → Application / Performance Service

The control-plane relationships operate in parallel with the user-plane path:

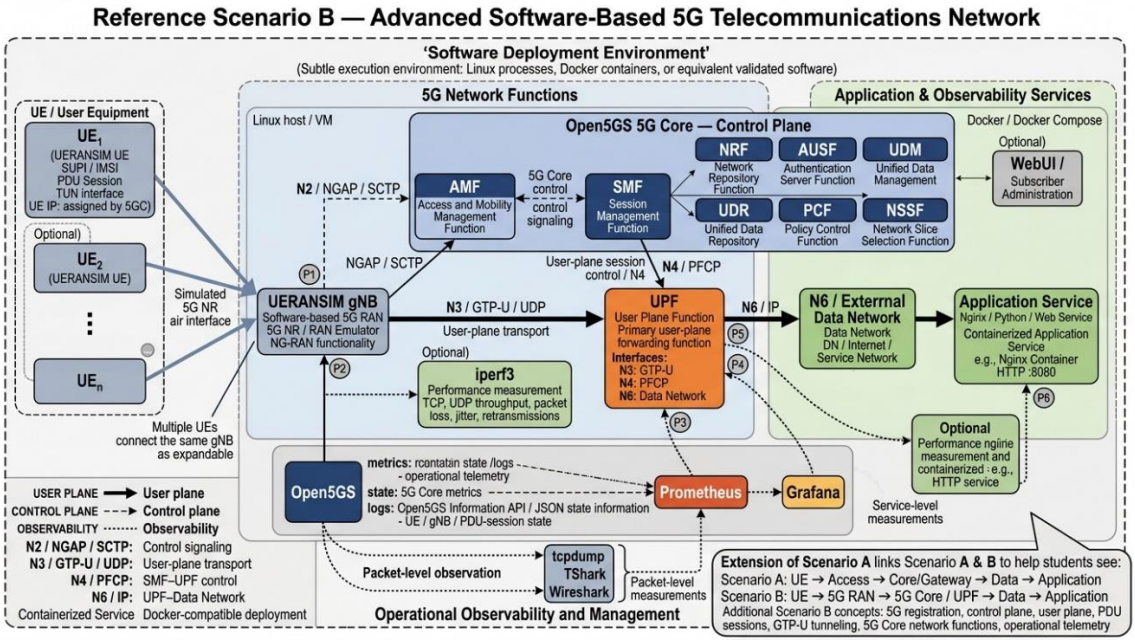
gNB → AMF → 5G Core control functions → SMF → UPF

and: **SMF → UPF via N4 / PFCP**

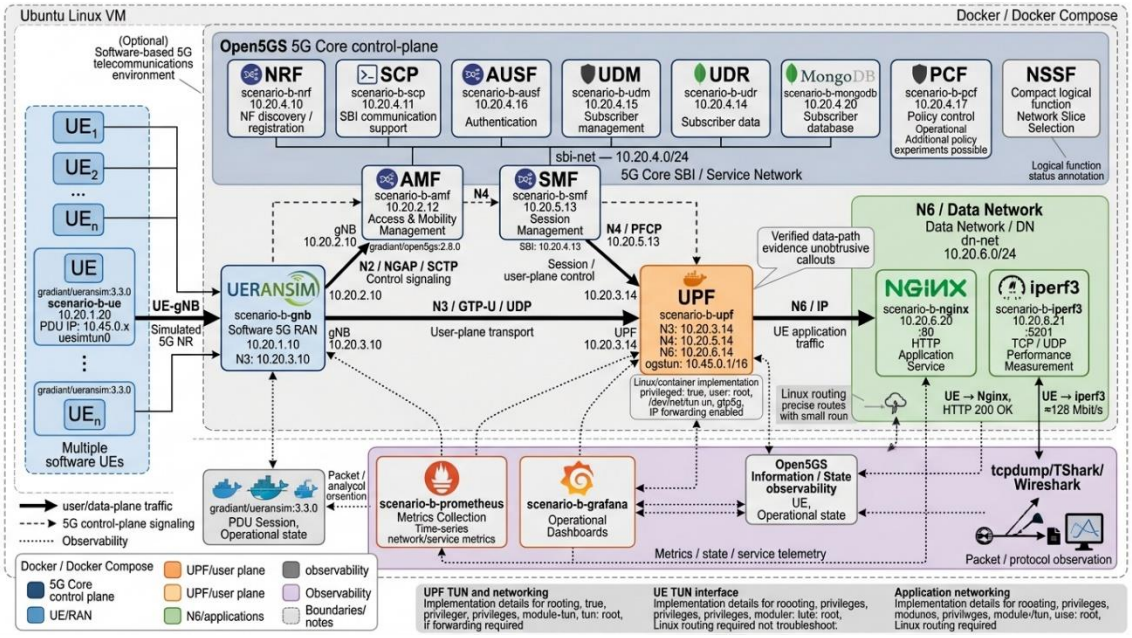
Operational information is observed separately through:

5G RAN / 5G Core / UPF / Services → Metrics / Logs / State / Packet Observation → Prometheus / Grafana / Wireshark

A recommended baseline topology is shown below (take it as a reference!)



Scenario Overview: Scenario B represents an advanced software-based telecommunications environment built around a 5G Standalone (SA) architecture. The scenario extends the Reference Scenario A by introducing: a software 5G UE; a software 5G RAN / gNB; 5G Core network functions; 5G registration and authentication; PDU-session establishment; explicit separation of control plane and user plane; N2 / NGAP / SCTP signaling; N3 / GTP-U user-plane transport; N4 / PFCP session control; N6 IP connectivity to a data network; an application service; a performance measurement service; network and service observability. The central end-to-end objective remains the same as in Scenario A: Establish communication, observe the communication, measure its behavior, introduce controlled changes, and diagnose failures using technical evidence. The AMF, SMF, NRF, AUSF, UDM, UDR, PCF, NSSF, and SCP are not application-payload forwarding nodes. The UPF is the principal user-plane forwarding function. The recommended implementation uses a Linux virtual machine with Docker and Docker Compose. The telecommunications functions are implemented as software processes running in containers. The practical deployment therefore consists of: UERANSIM, Open5GS Core Functions, MongoDB, Nginx, iperf3, etc. The deployment should be treated as one concrete realization of the 5G SA architecture. Different implementation choices are permitted, provided that the resulting system preserves the required functional relationships and can be demonstrated with technical evidence. A recommended Dockerized architecture is shown below.



Interface Mapping:

Interface	Purpose	Protocol	Typical endpoints
UE ↔ gNB	Simulated radio access	UERANSIM software interface	UE ↔ gNB
N2	RAN control signaling	NGAP / SCTP	gNB ↔ AMF
N3	User-plane transport	GTP-U / UDP	gNB ↔ UPF
N4	User-plane/session control	PFCP	SMF ↔ UPF
N6	Data-network connectivity	IP	UPF ↔ DN

Step 2: Traffic Observation and 5G Protocol Behavior

Use the deployed Scenario B network to investigate how control-plane signaling, user-plane traffic, and application traffic appear in the actual software network. The investigation must connect three levels of evidence: 5G procedure/network function → protocol and packet behavior → application/service behavior. Identify appropriate observation points in the Dockerized topology. At minimum, consider: P1 — UE / gNB side; P2 — N2: gNB ↔ AMF; P3 — N3: gNB ↔ UPF; P4 — N4: SMF ↔ UPF; P5 — N6: UPF ↔ Data Network; P6 — Application service; P7 — Performance service. The selected interfaces must be identified using actual Docker/Linux information. Use the UE and 5G Core logs to identify the major registration sequence. A representative procedure is: PLMN / cell selection → RRC establishment → Initial Registration → Authentication → Security Mode → Registration Accept → Registration Complete. Identify which network functions participate. At minimum, relate: UE; gNB; AMF; AUSF; UDM; UDR; MongoDB, where supported by the selected implementation. For each important procedure, record: originating function; receiving function; protocol/interface; evidence in logs; corresponding packet evidence where available. Other analyses include: N2 / NGAP / SCTP analysis (Analyze the gNB–AMF control relationship, explain why N2 traffic belongs to the control plane rather than the user plane), N3 / GTP-U analysis (Explain why the inner application packet can be carried inside GTP-U). N4 / PFCP analysis. Investigate the service-based architecture of the 5G Core. Relevant functions include: NRF; SCP; AMF; SMF; AUSF; UDM; UDR; PCF. Where evidence is available, identify source function; destination function; HTTP-based service interaction; service discovery; subscriber-data interaction; policy interaction. The objective is not to decode every SBI message. Instead, identify several representative interactions and explain their functional purpose. A useful conceptual grouping is NF discovery → service request → subscriber/authentication data → session/policy information. Do not imply that SBI traffic carries ordinary UE application payload. PDU-session procedure analysis includes UE logs+SMF logs+UPF logs+packet capture to identify the relationship between: PDU Session Establishment Request → SMF processing → PFCP / N4 → UPF session creation → UE IP assignment → uesimtun0 → N3 / GTP-U. The analysis should explain how a successful PDU session becomes an executable user-plane forwarding state. Others include N6 / application traffic analysis, TCP analysis (use either UE → Nginx or UE → iperf3), and UDP

analysis (Use iperf3 UDP traffic). Explain why UDP does not provide TCP-style transport-layer retransmission. Application-layer analysis: For the Nginx service, identify: HTTP request; HTTP response; status code; server endpoint; response time. Connect the application result to lower-layer evidence. A successful HTTP response should be interpreted as: the application is reachable and supported, where possible, by IP connectivity, TCP connection, packet capture, and 5G user-plane evidence.

Hints: Attention traps
Container state is not packet evidence; for example, docker compose ps shows process/container state. It does not prove that N2, N3, N4, and N6 are functioning. N2 = gNB ↔ AMF (control signaling), N3 = gNB ↔ UPF (user-plane GTP-U), N4 is not application traffic, N4 / PFCP controls UPF/session behavior. It does not carry ordinary HTTP data. GTP-U encapsulates entire subscriber IP packets—known as the inner IP packet—inside a UDP and outer IP wrapper to move data safely across mobile core networks. Docker IP is not automatically the UE PDU-session IP. A successful ping is not sufficient, as it only proves a particular ICMP path works. It does not automatically prove that HTTP, TCP, UDP, GTP-U, PFCP, and registration all work. N6 connectivity is not proof of 5G user-plane connectivity. For example: Alpine → Nginx only tests the Docker data network. UE → Nginx tests the complete 5G user-plane path. A packet visible on N6 is not identical in appearance to the same application traffic observed on N3, because N3 may contain GTP-U encapsulation.

Step 3: Performance Measurement, Network Degradation, and Data-Driven Network Analysis
Establish a quantitative performance baseline for the complete Scenario B 5G SA user-plane path and systematically investigate how controlled network conditions affect network performance, transport behavior, application performance, and observable network state. The experiment shall use the deterministic communication path: UE (10.45.0.x) → UERANSIM gNB (N3 / GTP-U / UDP) → Open5GS UPF (N6 Data Network 10.20.6.0/24, which contains Nginx/iperf3). The corresponding control and session-management context is: gNB (N2 / NGAP / SCTP) → AMF → SMF (N4 / PFCP) → UPF. Performance experiments should focus on the user-plane path, while control-plane state and operational telemetry should be used to explain changes in the measured behavior. The investigation shall establish the relationship: Configured network condition → Observed packet behavior → Transport behavior → Application performance → Operational telemetry → Quantitative interpretation.

Before introducing any artificial degradation, establish a baseline under normal operating conditions. The baseline should be measured using the complete path: UE → gNB → N3 → UPF → N6 → Application/Measurement Service. At least three repetitions should be performed for each important measurement. The baseline shall include the following categories. 1. Network-level performance, where measured: ICMP RTT; packet loss; packet count; minimum, average, maximum, and variation of RTT where appropriate. For example: docker exec scenario-b-ue ping -c 20 10.20.6.20. 2. TCP performance: use the iperf3 server: 10.20.6.21:5201. Measure: transfer volume; sender throughput; receiver throughput; retransmissions; test duration. Example: docker exec scenario-b-ue iperf3 -c 10.20.6.21 -t 10. 3. UDP performance: Run an iperf3 UDP experiment using a controlled offered bitrate. Measure: offered bitrate; received bitrate; packet loss; jitter; packet count, where available. The offered bitrate must be recorded because UDP results depend strongly on the sending rate. 4. Application performance: Use the Nginx endpoint: 10.20.6.20:80. Measure: HTTP success/failure; response time; TCP connection establishment; response size; repeated-request consistency. The exact command may be adapted to the available tools. The analysis should distinguish three measurement layers. Level 1 — Network (RTT; Packet loss; Packet timing, etc.), Level 2 — Transport (TCP throughput; TCP retransmissions; UDP throughput; UDP loss; UDP jitter), Level 3 — Application (HTTP response time; HTTP success rate; Service availability; Transfer completion time).

Some experiments:

1. Controlled Delay: Introduce a controlled delay using Linux traffic-control mechanisms such as tc/netem. The delay shall be applied at one explicitly identified measurement point. Examples: N3 / gNB–UPF or N6 / UPF–Data Network. A suitable first experiment is: Baseline + 50 ms additional delay. Measure: RTT; TCP throughput; TCP retransmissions; UDP throughput; UDP jitter; HTTP response time. Compare the

measured values with the baseline. The analysis includes configured delay → measured RTT change → transport response → application response.

2. Controlled Packet Loss: Introduce a controlled packet-loss condition. A suitable first experiment is: 1% packet loss at one selected location. Measure: TCP throughput; TCP retransmissions; UDP received bitrate; UDP loss; UDP jitter; HTTP response time and success rate. Compare TCP and UDP under the same network conditions. The expected conceptual distinction is: TCP packet loss → ACK/retransmission behavior → congestion response → possible throughput reduction, whereas UDP packet loss → datagrams may be lost → no TCP-style retransmission mechanism.

3. Controlled Rate Limitation: Apply a bandwidth (20 Mbps) limit at a selected user-plane segment. Measure: TCP throughput; TCP retransmissions; UDP throughput; UDP loss/jitter; HTTP response time. Discuss the effects of protocol overhead; TCP congestion control; packet size; measurement interval; Docker networking; Linux scheduling; virtual-machine execution; background traffic. A measured throughput below the configured rate should therefore not automatically be interpreted as a configuration error.

4. N3 versus N6 Performance Degradation: This experiment is particularly important for Scenario B because the architecture explicitly separates N3: gNB ↔ UPF from N6: UPF ↔ Data Network. Apply comparable impairments separately to N3 and N6. For example: Experiment N3: +50 ms delay; Experiment N6: +50 ms delay or: Experiment N3: 1% loss; Experiment N6: 1% loss. Compare: RTT; TCP throughput; TCP retransmissions; UDP throughput; UDP loss; UDP jitter; HTTP response time; service availability. The analysis should determine whether the location of the impairment changes the observed system behavior. The interpretation must distinguish the gNB–UPF user-plane segment from the UPF–Data Network segment rather than treating the entire user plane as one undifferentiated link.

5. Network Condition to Application Impact: Use repeated HTTP requests to the Nginx service while applying controlled network degradation. The experiment shall investigate: Network impairment → RTT / packet behavior → TCP behavior → HTTP response time → application availability. Measure: HTTP response time; successful requests; failed requests; TCP connection behavior; relevant packet-level evidence. Prometheus-Based Operational Measurement: Where Prometheus is deployed, use it as a time-series metrics system, not as an application traffic path. The measurement architecture is: 5G RAN / Open5GS / UPF / Services (metrics) → Prometheus (queries) → Grafana. Prometheus should be used to collect and retain time-series measurements, such as network-function counters; packet/traffic counters; session counters; service metrics; system/resource metrics; performance measurements. Grafana should be used primarily for visualization and comparison of time-series data collected from Prometheus or another documented data source. A suitable dashboard should contain logically related panels such as: Panel 1 — Throughput; Panel 2 — RTT / latency; Panel 3 — Packet loss; Panel 4 — TCP retransmissions; Panel 5 — UE / PDU-session state; Panel 6 — Application response time. The dashboard should make the temporal relationship between network condition and performance degradation/recovery visually identifiable. Grafana should not be represented as a packet-forwarding or user-plane component. For Packet Capture Correlation: At least one performance experiment shall be supported by packet-level evidence. Measured: TCP throughput decreases should be investigated against packet evidence such as: RTT increases; TCP retransmissions increase; duplicate ACKs appear; packet loss is observed where appropriate. Similarly, measured UDP loss increases should be supported by packet or iperf3 evidence. Packet capture must be interpreted according to the observation point. For N3, you should consider the presence of: Outer IP → UDP → GTP-U → Inner IP → TCP / UDP, while N6 generally exposes the IP/application side of the communication without the same GTP-U encapsulation.

Summary:

Students must deploy and verify a complete, reproducible software-based telecommunications network with the required network functions, interfaces, IP connectivity, routing, and end-to-end communication path, and demonstrate how the system operates through packet-level, network-function/system-level, and application-level observation. For Scenario A, the required communication path is UE/Client → Access Network → Core Network/Gateway → Data Network → Application Service, while for Scenario B, the required user-plane path is UE → gNB → N3/GTP-U → UPF → N6 → Data Network → Application Service, supported by the control-plane path gNB → N2/NGAP/SCTP → AMF → SMF → N4/PFCP → UPF and the relevant 5G service-based interfaces. Students must establish a repeatable performance baseline and quantitatively measure key indicators such as latency/RTT, packet loss, throughput,

retransmissions, jitter, and application response, then investigate controlled network degradation, including delay, packet loss, rate limitation, different impairment locations, and TCP/UDP behavior. Measurements must be collected into a structured dataset and analyzed using Python, NumPy, pandas, and Matplotlib, supported by meaningful technical visualizations and appropriate statistical summaries. Students must use Wireshark/TShark/tcpdump, network-function logs, Prometheus, and Grafana to correlate packet, system, and time-series evidence, while clearly distinguishing the purpose and limitations of each observation method. At least one abnormal condition or failure must be diagnosed through an evidence chain linking the observed symptom, affected path or component, packet/system/application evidence, measured impact, root cause, and recovery or correction. **All implementation materials must be pushed to a GitHub repository**. The exact structure may differ, but the repository must make it possible to understand how the network was deployed → how it was tested → how measurements were collected → how results were analyzed. The README may include: project/scenario description; system requirements; deployment instructions; verification commands; measurement procedure; impairment procedure; data-analysis procedure; expected outputs; known limitations.